

IPv4-64: Security and Performance Analysis

A technical comparison of IPv4-64 security mechanisms and forwarding performance against IPv4, IPv6, and prior address extension proposals

Registry: [[HOWL-NET-2-2026](#)]

Series Path: [[HOWL-NET-1-2026](#)] → [[HOWL-NET-2-2026](#)]

DOI: 10.5281/zenodo.23105515

Date: October 2026

Domain: Network Protocol Design / Security Analysis / Forwarding Performance

Status: Proposal — Not submitted to any standards body. Published for public review and discussion.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.6 Opus.

1. Purpose

This document analyzes the security mechanisms and forwarding performance of IPv4-64 as defined in the companion specification [[HOWL-NET-1-2026](#)]. It compares IPv4-64 feature-by-feature against IPv4 (RFC 791), IPv6 (RFC 8200), QUIC (RFC 9000), and three prior IPv4 address extension proposals: IPv4+4 (Valkó/Turányi, 2002), EnIP (draft-chimiak-enhanced-ipv4), and IPv10 (draft-omar-ipv10).

The analysis covers six areas: source address spoofing, SYN flood defense, fragment injection, UDP amplification, scanner reconnaissance, and forwarding pipeline performance in both hardware (ASIC) and software implementations.

This document does not discuss adoption strategy, deployment timelines, or organizational incentives. It examines only the technical properties of the protocol.

2. Background

2.1 What IPv4-64 Is

IPv4-64 is a network protocol that extends IPv4 addresses from 32 bits to 64 bits. It uses a new fixed-size IP header (32 bytes), a new fixed-size TCP header (24 bytes), and a new fixed-size UDP header (10 bytes). All three headers contain no variable-length fields. Every field is at a known byte offset.

The protocol keeps dotted-decimal address notation. Every existing IPv4 address is a valid IPv4-64 address with the upper 32 bits set to zero.

The full specification is in the companion document [[HOWL-NET-1-2026](#)]. This document assumes the reader has access to that specification but does not require prior reading. Each mechanism is explained before it is analyzed.

2.2 What Problem This Addresses

IPv6 introduced several features that the internet requires: a Flow Label for load balancing, removal of the per-hop header checksum, and mandatory UDP checksums. IPv6 also introduced features that created operational problems: an unbounded extension header chain, source-only fragmentation, and 128-bit addresses that are larger than necessary.

IPv4-64 takes the features that work (Flow Label, no header checksum, mandatory UDP checksum) and applies them to an IPv4-compatible design. It adds security mechanisms (source validation, fragment protection, SYN flood defense, amplification mitigation) that neither IPv4 nor IPv6 provide. It removes all variable-length parsing from the IP, TCP, and UDP headers.

The result is a protocol that is backward compatible with IPv4 addressing, incorporates the useful advances from IPv6 and QUIC, and adds new security and performance properties that no existing protocol provides.

2.3 Prior IPv4 Extension Proposals

Three prior proposals attempted to extend IPv4 address space without adopting IPv6.

IPv4+4 (Valkó/Turanyi, 2002): Creates 64-bit addresses by concatenating a 32-bit public gateway address with a 32-bit private host address. The architecture depends on NAT at every boundary gateway. It uses a modified version of the IPv4 Loose Source Record Route (LSRR) option to carry the extended address. It adds no security features, no flow labeling, and no transport header changes.

EnIP (draft-chimiak-enhanced-ipv4, 2015): Uses IPv4 options to carry additional address bits. The base header remains standard IPv4. Extended source and destination addresses are placed in an IP option field. The address notation is dotted decimal with 8 octets. EnIP increases address space by a factor of 17.9 million per existing routable address. It adds no security features and inherits all of IPv4's variable-length parsing requirements.

IPv10 (draft-omar-ipv10, 2016–2020): Does not extend the address space. It allows IPv4-only hosts to communicate with IPv6-only hosts by placing both IPv4 and IPv6 addresses in the same packet header. It is a bridging protocol. The IETF draft expired in 2021 without adoption.

IPv4+4 and EnIP address only the address exhaustion problem. Neither modifies the TCP or UDP headers. Neither adds security mechanisms. Neither removes variable-length parsing. IPv4-64 differs from both by defining a complete new protocol version with redesigned headers at every layer.

3. Security Analysis

3.1 Source Address Spoofing

3.1.1 The Problem A source address is the field in the IP header that identifies who sent the packet. In IPv4 and IPv6, any device can write any source address into any packet. Nothing in the packet proves the source address is correct. An attacker can send packets that appear to come from a different device. This is called source spoofing.

Source spoofing enables three categories of attack: anonymous denial-of-service (the attacker's real address is hidden), reflection attacks (the attacker sends requests to a public service with the victim's address as the source, causing the service to flood the victim with responses), and connection hijacking (the attacker injects packets into an existing connection by guessing the source address and sequence number).

3.1.2 Current Defense: BCP 38 BCP 38 (RFC 2827) is a recommended practice published in the year 2000. It states that ISPs should configure their edge routers to drop outgoing packets whose source addresses do not belong to the ISP's assigned address ranges. This is called ingress filtering.

BCP 38 is voluntary. Each ISP must configure it independently. There is no mechanism for a destination network to verify whether the source network applied the filter. After 26 years, deployment is not universal. A single non-compliant ISP upstream of an attacker is sufficient to allow spoofed packets onto the internet.

Neither IPv4 nor IPv6 provides any in-packet mechanism to verify source address authenticity.

3.1.3 IPv4-64 Defense: Source Validation Token The IPv4-64 IP header contains a 32-bit field called the Source Validation Token (SVT). The first-hop router — the router directly connected to the sender's network — computes this value using HMAC-SHA256 over the source address and a periodically rotating epoch counter, keyed with a 128-bit secret held by the router. The router writes the SVT into the packet on egress.

A downstream firewall or destination host that knows the first-hop router's secret can verify the SVT. A valid SVT means: a router that holds the correct secret attested that the source address belongs to the network segment the packet came from. An invalid SVT means the packet is either spoofed or originated from a network that does not participate in SVT.

3.1.4 Key Exchange Without a Central Authority The SVT key exchange does not use a certificate authority. It does not use a public key infrastructure. It does not use any central registry.

Two network operators who peer with each other share a 128-bit SVT secret directly. The secret is exchanged through the same operational channels used for BGP session

configuration: a phone call, an encrypted email, or a configuration management system. The same two humans who configure the BGP shared secret on a peering session add a second shared secret for SVT.

No third party is involved. No third party can issue, revoke, or substitute the secret. No government can compel a certificate authority to issue a fraudulent SVT secret, because no certificate authority exists. No corporation can purchase trust by presenting incorporation documents, because there is no registration process. The only way to enter the trust circle is to establish a physical peering relationship and bilaterally negotiate a secret exchange.

This trust model is identical to how BGP operates today. BGP has no central authority. Operators trust their peers because they have direct physical relationships: fiber cross-connects in the same facility, router ports in the same cage, contracts negotiated between known individuals. The SVT extends this existing physical trust model to source address verification.

3.1.5 Incremental Deployment The SVT provides value without universal deployment. If two operators share SVT secrets, each can verify packets from the other. Packets from non-participating networks arrive with unverifiable SVTs. The receiving network's policy determines the response: pass, rate-limit, deprioritize, or drop.

The top 10 transit providers carry the majority of internet traffic. If those providers implement SVT stamping and share secrets bilaterally, every packet transiting between them has a verifiable source attestation. A spoofed packet entering from a non-participating network is identifiable by the receiving network.

The incentive is bilateral. If Operator A shares its SVT secret with Operator B, then Operator B can drop spoofed traffic claiming to come from Operator A's address space. This protects Operator A's reputation and reduces abuse complaints directed at Operator A. Both parties benefit from the exchange. This is a stronger incentive than BCP 38, which protects other networks but not the filtering network itself.

3.1.6 Residual Weakness The SVT is 32 bits. An attacker who does not know the secret can attempt to brute-force the value. 2^{32} is approximately 4.3 billion possibilities. On a high-bandwidth link, an attacker can send billions of packets per second. However, each attempt requires a full packet with a plausible source address, destination address, and correct epoch timing. The two-epoch overlap (current and previous secrets are both valid) provides a window of approximately $2N$ minutes (where N is the epoch duration). An attacker must hit the correct SVT within this window for a specific source address. Random flooding with random SVTs will produce a valid match at a rate of approximately 1 in 4.3 billion packets per source address per epoch.

For comparison: IPv6 does not provide any source validation mechanism. The 128-bit address space makes scanning harder (finding a valid target address requires more guesses), but provides no cryptographic proof that a packet originated from the claimed source.

3.1.7 Comparison

Property	IPv4	IPv6	IPv4+4	EnIP	IPv4-64
In-packet source proof	None	None	None	None	SVT (32-bit HMAC)
Spoofing defense	BCP 38 (voluntary, unverifiable)	BCP 38 (voluntary, unverifiable)	None specified	None specified	SVT (verifiable per packet)
Key distribution	N/A	N/A	N/A	N/A	Bilateral, no central authority
Destination can detect spoofing	No	No	No	No	Yes (if peer shares secret)

3.2 SYN Flood Defense

3.2.1 The Problem TCP connections begin with a three-way handshake. The client sends a SYN packet. The server responds with a SYN-ACK. The client responds with an ACK. The connection is established.

In a SYN flood attack, the attacker sends millions of SYN packets per second, often with spoofed source addresses. The server allocates a small amount of memory (a SYN queue entry) for each SYN it receives, waiting for the corresponding ACK. The ACK never arrives because the source address is fake. The SYN queue fills. Legitimate connections cannot be established.

3.2.2 Current Defense: SYN Cookies SYN cookies are an implementation technique used in IPv4 TCP stacks. Instead of allocating a SYN queue entry, the server encodes connection state into the 32-bit TCP sequence number of the SYN-ACK. The client echoes this value in its ACK. The server decodes the sequence number to recover the connection parameters and allocates state only after verification.

SYN cookies work, but the 32-bit sequence number is too small to carry all the required data. When SYN cookies are active, TCP option negotiation is lost. Window scaling, selective acknowledgment (SACK), and timestamps cannot be negotiated because there are not enough bits in the sequence number to encode them alongside the cookie.

In practice, many systems activate SYN cookies only during detected attacks. During normal operation, the SYN queue is used. During an attack, the system switches to SYN cookies and loses TCP features. This creates a degraded performance mode exactly when the network is under stress.

FreeBSD extended SYN cookies to store window scaling and SACK information in the TCP timestamp fields when timestamps are available. This partially recovers the lost

features but depends on the client supporting TCP timestamps, which is not universal.

3.2.3 QUIC Defense: Retry Token QUIC (RFC 9000) solved this problem by adding a dedicated Retry Token field to the handshake. The server sends a Retry packet containing a token computed from the client's address and a server secret. The client must echo the token in its next packet. The server verifies the token before allocating connection state.

The Retry Token is separate from the sequence number. No transport features are sacrificed. The mechanism is always active, not emergency-only. Google, Cloudflare, and Fastly operate QUIC at scale with this model.

3.2.4 IPv4-64 Defense: Retry Cookie The IPv4-64 TCP header contains a dedicated 64-bit Retry Cookie field. On a SYN packet, this field is zero. On a SYN-ACK, the server fills it with a value computed from HMAC-SHA256 over the client's address, port, server secret, and a timestamp. The lower 8 bits of the cookie encode the negotiated TCP parameters: MSS (3 bits), window scale factor (4 bits), and SACK capability (1 bit).

The client echoes the cookie in its ACK. The server verifies the cookie, recovers the TCP parameters from the lower 8 bits, and allocates connection state. If the ACK never arrives, zero state was allocated. If the ACK carries an invalid cookie, the packet is silently dropped and zero state is allocated.

This mechanism is always active. There is no emergency mode. There is no performance penalty. Window scaling and SACK are preserved during flood conditions.

3.2.5 Combined Effect: SVT + Retry Cookie When both mechanisms are active, a SYN flood attack faces two layers of defense.

Layer 1: If the attacker uses spoofed source addresses, the SVT is invalid. Packets are dropped before they reach the TCP stack. The server never processes the SYN.

Layer 2: If the attacker uses real source addresses (the SVT is valid because the attacker's first-hop router stamped it), the server sends a SYN-ACK with a Retry Cookie. The attacker must receive the SYN-ACK at the real source address and echo the cookie back. This forces the attacker to complete a real handshake, which removes anonymity and rate-limits the attack to the attacker's actual connection capacity.

The attacker can no longer flood anonymously. Either the packets are dropped at Layer 1 (spoofed source), or the attacker must reveal their real address and complete real handshakes at Layer 2 (valid source).

3.2.6 Comparison

Property	IPv4 SYN Cookies	QUIC Retry Token	IPv4-64 Retry Cookie
Field location	32-bit sequence number (overloaded)	Dedicated field	Dedicated 64-bit field
Window scaling preserved	No	Yes	Yes (4 bits in cookie)
SACK preserved	No	Yes	Yes (1 bit in cookie)
Activation mode	Emergency only (most systems)	Always active	Always active
Cookie strength	32 bits	Variable	64 bits
Combined with source validation	No	No	Yes (SVT + Retry Cookie)

3.3 Fragment Injection

3.3.1 The Problem When a packet is larger than the maximum transmission unit (MTU) of a link on its path, it is broken into smaller pieces called fragments. Each fragment carries an Identification value that groups it with the other fragments of the same original packet. The receiver reassembles fragments with the same Identification into the original packet.

In a fragment injection attack, an off-path attacker guesses the Identification value of a legitimate fragmented packet and sends a forged fragment with the same Identification but different content. The receiver cannot distinguish the forged fragment from the legitimate one. It reassembles the forged content into the packet. This can corrupt data, inject malicious payloads, or crash the receiver.

3.3.2 IPv4 Defense IPv4 provides no defense against fragment injection. The Identification field is 16 bits. An attacker must guess a value from 65,536 possibilities. On a fast link, an attacker can cycle through all 65,536 values in milliseconds.

3.3.3 IPv6 Defense IPv6 prohibits transit routers from fragmenting packets. Only the source can fragment. This means an off-path attacker cannot inject fragments into a flow between two hosts unless the source is already fragmenting. The attack surface is reduced.

The tradeoff is that path MTU discovery must work correctly. Path MTU discovery relies on ICMP “Packet Too Big” messages from routers that encounter a packet larger than their link MTU. If ICMP is blocked (which is common in enterprise firewalls), the sender never learns the path MTU. Packets are silently dropped. The connection stalls. This is called a PMTUD black hole. It is a known operational problem with IPv6.

3.3.4 IPv4-64 Defense: Fragment Token The IPv4-64 IP header contains a 16-bit Fragment Token. The source computes this value using SipHash-2-4 over the Identification field, source address, destination address, and a per-connection secret. All fragments of the same original packet carry the same Fragment Token. The receiver drops fragments whose tokens do not match.

An off-path attacker must guess both the 16-bit Identification and the 16-bit Fragment Token simultaneously. This is 2^{32} combinations (approximately 4.3 billion), which is 65,536 times harder than IPv4.

IPv4-64 allows both source and transit routers to fragment. This preserves operational flexibility. A transit router that encounters a packet larger than the link MTU can fragment it. Path MTU discovery is not required. PMTUD black holes do not occur.

3.3.5 Comparison

Property	IPv4	IPv6	IPv4-64
Fragment injection difficulty	2^{16} (65,536 guesses)	Reduced (source-only fragmentation)	2^{32} (4.3 billion guesses)
Transit fragmentation allowed	Yes	No	Yes
PMTUD black hole risk	Low (transit fragmentation available)	High (when ICMP blocked)	None (transit fragmentation available)
On-path attacker defense	None	None	None (by design — on-path requires encryption)

3.4 UDP Amplification

3.4.1 The Problem Some UDP services send large responses to small requests. A DNS query is approximately 60 bytes. The response can be over 4,000 bytes. An NTP monlist request is 234 bytes. The response can be over 48,000 bytes. A memcached stats request is a few bytes. The response can exceed 1 megabyte.

An attacker sends a small request to one of these services with the victim's address as the source. The service sends the large response to the victim. The attacker sends requests to thousands of services simultaneously. The victim receives a flood of responses it never requested. The amplification factor (response size divided by request size) can exceed 50,000x for memcached.

3.4.2 IPv4 and IPv6 Defense Neither IPv4 nor IPv6 provides any in-packet mechanism to mitigate UDP amplification. The current defenses are application-specific: DNS Response Rate Limiting, NTP monlist disabling, memcached access controls. Each application must implement its own defense. A middlebox (firewall, scrubbing service) that wants to throttle amplification traffic must understand each application protocol individually.

3.4.3 IPv4-64 Defense: Validated Flag The IPv4-64 UDP header contains a 1-bit Validated flag. A UDP server sets this flag when it has verified the source through application-layer challenge-response (such as a DNS cookie exchange) or through SVT verification.

A middlebox on the network path reads the Validated flag at a fixed byte offset. If the flag is not set, the middlebox can rate-limit the response. This throttles amplification traffic at the network layer without the middlebox needing to understand DNS, NTP, memcached, or any other application protocol.

3.4.4 Combined Effect: SVT + Validated Flag The spoofed request arrives at the UDP service. If the source network participates in SVT, the request's SVT is invalid (the spoofed source address does not match the stamping router's network). The request is dropped before it reaches the service. The amplification never occurs.

If the source network does not participate in SVT (partial deployment), the request reaches the service. The service sends a response. The response does not have the Validated flag set (no application-layer source verification occurred). The middlebox on the victim's ingress link rate-limits the unvalidated response.

The source-side defense (SVT) works when the source network participates. The victim-side defense (Validated flag) works regardless of source-network participation. Both defenses operate at fixed byte offsets with no application-layer parsing.

3.4.5 Comparison

Property	IPv4	IPv6	IPv4-64
Generic amplification defense	None	None	Validated flag (1 bit)
Application-specific defense required	Yes (per protocol)	Yes (per protocol)	No (Validated flag is protocol-agnostic)
Combined with source validation	No	No	Yes (SVT blocks spoofed requests)
Middlebox application awareness required	Yes	Yes	No

3.5 Scanner Reconnaissance

3.5.1 The Problem Before attacking a host, an attacker must discover that the host exists and identify what services it runs. A scanner sends packets to a target address. If the host is alive, it responds. An ICMP echo reply confirms the host exists. A TCP RST on a closed port confirms the host exists and that the port is closed. An ICMP Port Unreachable on a UDP probe confirms the host exists. Each response leaks information.

The responses also enable operating system fingerprinting. Different operating systems generate different TTL values, TCP window sizes, and ICMP response formats. An attacker who receives a response can often identify the operating system without ever connecting to an open port.

3.5.2 IPv4 and IPv6 Defense Both IPv4 and IPv6 generate error responses to invalid or undeliverable packets. ICMP Destination Unreachable, ICMP Time Exceeded, TCP RST, and ICMPv6 equivalents all confirm that a host is alive and reachable.

Operators can configure firewalls to suppress some of these responses, but this is a per-host or per-network configuration decision. The protocols themselves specify that error responses should be generated.

3.5.3 IPv4-64 Defense: Strict Drop IPv4-64 specifies that a malformed, undeliverable, or unrecognized packet is silently dropped. No ICMP error is sent. No TCP RST is sent. No response of any kind is generated. The receiver processes the next packet.

The scanner receives nothing. It cannot distinguish a live host that silently dropped the packet from a nonexistent address. Host discovery requires finding an open service that responds to a valid request, which the attacker already needs for exploitation.

3.5.4 Diagnostics as a Separate Protocol IPv4 embedded diagnostic functions (ping, traceroute, path MTU discovery) into ICMP, which operates at the same layer as production traffic. Any host that responds to production traffic also responds to diagnostic probes. This was appropriate in 1981 when the network was small and trusted. In 2026, embedding diagnostics into the forwarding protocol means that every diagnostic response is also information leakage to an attacker.

IPv4-64's strict drop behavior means that diagnostics cannot operate through ICMP in the same way. Diagnostic functions (reachability testing, path tracing, MTU discovery) require a separate protocol with its own authentication and access controls. Authorized operators authenticate to the diagnostic protocol. Unauthorized parties receive no response. The diagnostic protocol can be restricted to management VLANs, specific source addresses, or authenticated sessions.

This separation is a stronger architecture. Production traffic reveals nothing about host existence. Diagnostic traffic is restricted to authorized sources. The two functions do not share a response channel.

3.5.5 Comparison

Property	IPv4	IPv6	IPv4-64
Response to invalid packets	ICMP error	ICMPv6 error	Silent drop
Host existence revealed	Yes	Yes	No
OS fingerprinting via responses	Yes	Yes	No
Diagnostics	ICMP (same channel as production)	ICMPv6 (same channel as production)	Separate authenticated protocol (not specified in base protocol)

3.6 Firewall Evasion

3.6.1 The Problem A firewall inspects packet headers to enforce security policies. It must read the transport protocol, source and destination addresses, source and destination ports, and TCP flags. If the firewall interprets a packet differently from the destination host, an attacker can craft packets that the firewall permits but the destination processes in a harmful way. This is called a parser differential attack.

Parser differentials arise from variable-length headers. The firewall and the destination must agree on where each field is located in the packet. If the header length is variable, they must both parse the header correctly to agree on field positions.

3.6.2 IPv4 Parser Differentials The IPv4 IHL field determines the header length (20–60 bytes). The transport header begins at byte offset $\text{IHL} \times 4$. If IHL is set to an unusual value, the firewall and the destination may disagree on where the transport header starts. IPv4 options add further complexity: each option has a type-length-value structure. A malformed option (incorrect length field, overlapping options, options that extend past the header boundary) can cause one parser to skip to a different position than another.

This class of evasion was documented by Ptacek and Newsham in 1998. It remains relevant because IPv4 implementations still contain variable-length parsing logic.

3.6.3 IPv6 Parser Differentials IPv6 extension headers create the same problem at a larger scale. The firewall must walk the next-header chain to find the transport layer. An attacker can use long chains, unknown extension types, overlapping fragments in extension headers, or extension headers that span fragment boundaries. Different firewalls handle these cases differently. Some drop the packet. Some skip the unknown extension. Some pass the packet without inspecting the transport layer.

3.6.4 IPv4-64: No Parser Differentials Every IPv4-64 field is at a fixed byte offset. The transport protocol number is at byte 7. Source port is at byte 32 of the combined IP+transport header. Destination port is at byte 34. TCP flags are at byte 44. There is no IHL. There are no options. There are no extension headers. There is nothing to parse.

A firewall reads fixed offsets. The destination reads the same fixed offsets. There is no ambiguity. There is no variable-length header that two implementations might parse differently. Parser differential attacks are structurally impossible because there is no parser — only fixed-offset reads.

3.6.5 Comparison

Property	IPv4	IPv6	IPv4-64
Header length variable	Yes (IHL)	Yes (extension chain)	No (always 32 bytes)
Transport header offset variable	Yes	Yes	No (always byte 32)
Parser differential attacks possible	Yes	Yes	No (no parser exists)
Firewall field extraction	Requires IHL computation	Requires chain walk	Fixed-offset reads

3.7 Security Summary

The following features are present only in IPv4-64 and are not present in IPv4, IPv6, QUIC, IPv4+4, EnIP, or IPv10:

- Source Validation Token in the IP header, exchanged through bilateral trust without a central authority
- Fragment Token binding fragments to a per-connection secret
- Fixed TCP header with a dedicated Retry Cookie field that preserves window scaling and SACK during flood defense
- UDP Validated flag for generic, protocol-agnostic amplification mitigation
- TCP CV flag for wire-speed connection state discrimination without TCP state tables
- Silent drop as a protocol-level requirement, not an implementation choice
- Complete elimination of parser differential attacks through fixed-offset headers at all layers

Each mechanism provides partial defense individually. Together they close the three largest classes of volumetric attack on the current internet: source spoofing, SYN flood-

ing, and UDP amplification. No existing deployed protocol addresses all three at the network and transport layer.

4. Forwarding Performance Analysis

4.1 What Determines Forwarding Speed

A router receives a packet, reads the destination address, looks up the next hop in a routing table, decrements the time-to-live (TTL) counter, and sends the packet out the correct interface. The time this takes per packet determines the maximum forwarding rate.

At 400 Gbps line rate with 64-byte minimum-size packets, a router must process approximately 595 million packets per second. Each packet has approximately 1.68 nanoseconds of processing budget. Any per-packet operation that takes longer than this budget causes the router to fall below line rate.

The two factors that determine per-packet processing time are: how many operations the chip must perform per packet, and whether the operation sequence is the same for every packet (constant pipeline) or varies depending on packet content (variable pipeline).

4.2 Variable-Length Headers and Pipeline Depth

A network ASIC processes packets in a pipeline. Each stage of the pipeline performs one operation: read a field, compare a value, modify a value, perform a lookup. Each stage takes one clock cycle.

IPv4: The IHL field determines the header length (20–60 bytes). The chip must read IHL before it knows where subsequent fields are. This requires either additional pipeline stages that sit idle when headers are short (wasted silicon area and power), or multiplexers that select different data paths based on header length (additional gate count and propagation delay). The worst case is a 60-byte header with options requiring per-option type dispatch.

IPv6: The base header is fixed at 40 bytes. However, extension headers create variable processing depth. The chip reads the Next Header field and follows the chain until it finds a transport protocol. The chain length is unbounded. ASICs impose a limit (typically 2–4 extension headers). Packets exceeding the limit are sent to the CPU (slow path) for software processing. The slow path operates at orders of magnitude lower throughput.

IPv4-64: The header is 32 bytes. Every field is at a known offset. The pipeline is a fixed sequence of reads at constant offsets. No multiplexers. No idle stages. No branching. The worst-case processing time equals the best-case processing time. There is no slow path for header parsing.

4.3 Header Checksum Elimination

IPv4: Every router on the path recomputes the header checksum after decrementing TTL. The checksum is a 16-bit one's complement sum over the header bytes. On a 20-byte header, this is 10 addition operations. On a 60-byte header with options, this is 30

addition operations. At 595 million packets per second (400 Gbps with minimum-size packets), this is 1.19 billion checksum operations per second (verification on input plus recomputation on output).

IPv6: No header checksum. This was one of the performance improvements IPv6 introduced.

IPv4-64: No header checksum. The link layer (Ethernet CRC-32) detects transmission errors. The transport layer (TCP/UDP checksums) detects end-to-end corruption. The per-hop header checksum in IPv4 is redundant and consumes processing budget on every router. IPv4-64 removes it for the same reason IPv6 did.

4.4 ASIC Design Implications

A fixed pipeline depth means the chip designer knows the exact number of clock cycles per packet before the chip is fabricated. The pipeline is designed for exactly one processing time. No silicon area is spent on handling variable-length edge cases. No transistors are spent on option-parsing state machines. No power is spent on idle pipeline stages.

The gate count reduction is significant. An IPv4 header parser must handle 11 possible header lengths (IHL values 5 through 15), option type dispatch, and variable checksum width. An IPv4-64 header parser reads 32 bytes from fixed offsets. The logic is combinational (fixed wiring), not sequential (state machine). Combinational logic is faster, uses less power, and occupies less silicon area than sequential logic.

The practical effect: a given ASIC fabrication process (transistor size, supply voltage, clock frequency) yields a higher maximum forwarding rate with IPv4-64 than with IPv4 or IPv6. Alternatively, it yields the same forwarding rate at lower power consumption, or the same forwarding rate on a smaller (cheaper) die.

4.5 Software Router Performance

4.5.1 Branch Prediction Modern CPUs predict which path a conditional branch will take before the condition is evaluated. A correct prediction costs nothing. A mispredicted branch costs 10–20 clock cycles (the pipeline must be flushed and refilled).

IPv4 packet receive path (Linux kernel): The `ip_rcv` function checks IHL, total length, header checksum, and version. If options are present, `ip_options_compile` parses them in a loop with a switch statement on option type. Each option type is a branch. The branch predictor cannot reliably predict which option types will appear because options are rare and varied. Each misprediction costs 10–20 cycles.

IPv4-64 packet receive path: No option parsing. No switch statement. No loop. The receive function performs: one comparison (Version), one comparison (Payload Length against received bytes), one subtraction (TTL decrement), one lookup (route table on destination address). The function has two conditional branches (version check and length check), both of which are almost always taken (valid packets vastly outnumber invalid ones). The branch predictor achieves near-100% accuracy.

4.5.2 Memory Allocation IPv4: If TCP options are present, the kernel allocates an `ip_options` structure on the heap. Each heap allocation involves a call to the memory allocator, which may involve lock contention on multi-core systems and is a potential cache miss (the allocated memory may not be in the CPU cache).

IPv4-64: No option structures exist. No heap allocation is required for header processing. All header fields are read from the packet buffer, which is already in the CPU cache from the network interface DMA transfer.

4.5.3 Cache Behavior A CPU cache line is typically 64 bytes. Reading data that is not in the cache requires fetching it from a slower level of memory. An L1 cache hit costs approximately 1 nanosecond. An L2 cache hit costs 4–5 nanoseconds. A main memory access costs 50–100 nanoseconds.

IPv4: The IP header is 20–60 bytes. A 20-byte header fits in one cache line with room for transport data. A 60-byte header with options may span two cache lines. The TCP header adds 20–60 bytes. In the worst case (60-byte IP header + 60-byte TCP header = 120 bytes), the headers span two or three cache lines.

IPv4-64: The IP header is 32 bytes. The TCP header is 24 bytes. Total: 56 bytes. This fits in one 64-byte cache line with 8 bytes remaining. One cache line read gives the router or host the complete IP header and the complete TCP header. One memory access for all header processing.

4.6 TCP Per-Segment Processing

In an established TCP connection, every data segment carries a TCP header. The receiver must process this header to extract the sequence number, acknowledgment number, window size, and flags.

IPv4 TCP: The receiver must also scan the TCP options region on every segment. The common case is a 12-byte options region containing a NOP padding byte and a 10-byte timestamp option. The receiver's TCP stack iterates over the options to find the timestamp. This is a small loop (typically 2 iterations), but it executes on every received segment. At 10 million segments per second, this is 20 million loop iterations and 20 million branch predictions per second.

IPv4-64 TCP: No options region exists. The sequence number, acknowledgment number, window size, and flags are all at fixed offsets. The Retry Cookie field is zero on data segments (not during handshake). There is no per-segment option scanning. The receiver reads fixed offsets only.

4.7 Firewall Processing

A firewall extracts a 5-tuple (source address, destination address, source port, destination port, protocol) from each packet and matches it against a rule table.

IPv4 firewall: The firewall must compute the transport header offset from IHL. It must parse the TCP Data Offset to find TCP flags. The 5-tuple extraction requires reading IHL,

computing the offset, then reading the transport fields. On packets with IP options, the firewall may need to parse the options to determine if source routing is present (which changes the effective destination address).

IPv6 firewall: The firewall must walk the extension header chain to find the transport header. It must handle all extension header types it recognizes and make a policy decision about types it does not recognize.

IPv4-64 firewall: Protocol is at byte 7. Source address is at bytes 16–23. Destination address is at bytes 24–31. Source port is at bytes 32–33 of the full packet. Destination port is at bytes 34–35. TCP flags are at bytes 44–45. Five fixed-offset reads. No computation. No parsing.

This maps directly to TCAM (Ternary Content-Addressable Memory) hardware in network appliances. A TCAM matches a fixed-width key against a table of rules in one clock cycle. IPv4 and IPv6 require a pre-processing stage to normalize variable-length headers into a fixed-width TCAM key. IPv4-64 requires no pre-processing because the header is already fixed-width.

4.8 Overhead Comparison

4.8.1 Header Size on Bulk Data On a 1500-byte MTU with bulk TCP data transfer:

Stack	IP + TCP Headers	Header % of 1500-byte Frame
IPv4 (minimum)	40 bytes	2.67%
IPv6	60 bytes	4.00%
IPv4-64	56 bytes	3.73%

IPv4-64 is 4 bytes smaller than IPv6 per packet. On bulk data, the difference is negligible.

4.8.2 Header Size on Small Packets On small packets (VoIP, gaming, real-time telemetry, API calls), header overhead is significant because the payload is small relative to the header.

Stack	IP + UDP + 64-byte Payload	Header % of Total
IPv4	92 bytes	30.4%
IPv6	112 bytes	42.9%
IPv4-64	106 bytes	39.6%

IPv4-64 is 6 bytes smaller than IPv6 on small UDP packets. On a VoIP call generating 50 packets per second per direction, this saves 600 bytes per second per call. On a system handling 100,000 concurrent VoIP calls, this saves 60 megabytes per second of bandwidth compared to IPv6.

4.9 Performance Summary

Property	IPv4	IPv6	IPv4-64
ASIC pipeline depth	Variable (IHL-dependent)	Variable (extension chain)	Constant
Worst case equals best case	No	No	Yes
Per-hop checksum computation	Yes	No	No
Cache lines for IP + TCP headers	1–2	1–2	1
Per-segment TCP option parsing	Yes	Yes (same TCP)	None
Slow-path punt for unusual headers	Yes (options)	Yes (long extension chains)	Never
Firewall field offsets constant	No	No	Yes
TCAM key extraction requires pre-processing	Yes	Yes	No
Software branches per packet (header parsing)	Multiple (IHL, options loop, checksum)	Multiple (extension chain walk)	Two (version check, length check)
Heap allocation for header processing	Yes (options structure)	Yes (extension header chain)	No

5. Comparison to Prior Address Extension Proposals

IPv4+4 and EnIP are address-only extensions. They add address space but do not modify the TCP or UDP headers, do not add security mechanisms, and do not remove variable-length parsing. The following table compares all features across proposals.

Feature	IPv4	IPv6	IPv4+4	EnIP	IPv10	IPv4-64
Address width	32 bits	128 bits	64 bits	64 bits	Existing (bridge)	64 bits
Address notation	Dotted decimal	Colon hex	Two dotted-decimal halves	Dotted decimal (8 octets)	Mixed	Dotted decimal (4–8 octets)
Wire format	IPv4 header	New header	IPv4 with LSRR option	IPv4 with extended option	New header	New fixed header

Feature	IPv4	IPv6	IPv4+4	EnIP	IPv10	IPv4-64
Requires NAT	No (but used)	No	Yes (architecture depends on NAT)	Yes (stateless EnIP NAT)	No	No
Variable-length IP parsing	Yes (IHL + options)	Yes (extension chain)	Yes (IPv4 options)	Yes (IPv4 options)	Yes	No
Variable-length TCP parsing	Yes (Data Offset + options)	Yes (same TCP)	Yes (same TCP)	Yes (same TCP)	Yes	No
Per-hop header checksum	Yes	No	Yes	Yes	Yes	No
Flow Label	No	Yes	No	No	No	Yes
Source Validation	No	No	No	No	No	Yes
Token Fragment	No	No	No	No	No	Yes
Token SYN flood defense (dedicated field)	No	No	No	No	No	Yes (Retry Cookie)
UDP amplification defense	No	No	No	No	No	Yes (Validated flag)
Silent drop semantics	No	No	No	No	No	Yes
Fixed pipeline depth	No	No	No	No	No	Yes
ASIC worst case = best case	No	No	No	No	No	Yes

Feature	IPv4	IPv6	IPv4+4	EnIP	IPv10	IPv4-64
New DNS record type	No	AAAA	No	No	No	AA
Existing IPv4 addresses valid	Yes	No	Yes (lower 32 bits)	Yes (site address)	Partially	Yes (upper bits zero)
Transport header changes	N/A	None (same TCP/UDP)	None	None	None	TCP and UDP re-designed

6. Conclusion

IPv4-64 provides three categories of improvement over existing protocols.

Security: Six in-header mechanisms (SVT, Fragment Token, Retry Cookie, Validated flag, CV flag, strict drop) that interact to close the three largest classes of volumetric internet attack. The SVT key exchange operates through bilateral trust without a central authority, using the same physical trust relationships that already exist in network peering.

Performance: Constant pipeline depth, no per-hop checksum, single-cache-line headers, no per-segment option parsing, no heap allocation, branchless header processing, and direct TCAM key extraction without pre-processing.

Compatibility: Every IPv4 address is valid. Dotted-decimal notation is preserved. Existing DNS A records are unchanged. Existing firewall rule notation is unchanged. The transition requires no dual-stack, no translation gateways, and no parallel infrastructure.

No existing deployed protocol or prior proposal provides all three categories simultaneously.

References

- RFC 791: Internet Protocol, September 1981
- RFC 2827 (BCP 38): Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing, May 2000
- RFC 8200: Internet Protocol, Version 6 (IPv6) Specification, July 2017
- RFC 9000: QUIC: A UDP-Based Multiplexed and Secure Transport, May 2021

- Valkó, A., Turányi, Z., “4+4: An Alternative Approach to IPv4 Address Exhaustion,” SIGCOMM Computer Communication Review, 2002
- draft-chimiak-enhanced-ipv4-03: IPv4 with 64 bit Address Space (EnIP), expired 2016
- draft-omar-ipv10-13: Internet Protocol version 10 (IPv10), expired 2021
- Ptacek, T., Newsham, T., “Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection,” Secure Networks Inc., 1998
- [HOWL-NET-1-2026]: IPv4-64: A 64-Bit In-Place Upgrade to IPv4 with Modern Security and Performance, October 2026

Appendix A — Attack Surface Comparison Matrix

This table maps each attack class to the specific header field or mechanism that enables or prevents it in each protocol. A dash means the protocol provides no relevant mechanism.

Attack Class	Attack Mechanism	IPv4 Enabler	IPv4 Defense	IPv6 Enabler	IPv6 Defense	IPv4-64 Enabler	IPv4-64 Defense
Source spoofing	Forged source address	Source address is unsigned	BCP 38 (voluntary)	Source address is unsigned	BCP 38 (voluntary)	—	SVT (32-bit HMAC at byte 12)
Reflection amplification	Spoofed request to public service	No source verification + optional UDP checksum	—	No source verification	—	—	SVT (source side) + Validated flag (victim side)
SYN flood	State exhaustion via half-open connections	SYN queue allocation before validation	SYN cookies (overloads sequence number)	Same as IPv4	Same as IPv4	—	Retry Cookie (dedicated 64-bit field, zero state until verified)

Attack Class	Attack Mechanism	IPv4 Enabler	IPv4 Defense	IPv6 Enabler	IPv6 Defense	IPv4-64 Enabler	IPv4-64 Defense
Fragment injection	Forged fragment with guessed Identification	16-bit Identification (65,536 guesses)	—	Source-only fragmentation (reduces surface)	PMTUD dependency	—	Fragment Token (16-bit keyed hash, 2^{32} combined guesses)
Fragment overlap	Overlapping offsets produce ambiguous re-assembly	No overlap detection required by spec	Implementation dependent	RFC 8200 Section 4.5 drops overlapping fragments	Specified	—	Drop entire fragment group (specified)
Scanner reconnaissance	Error responses reveal host existence	ICMP Destination Unreachable, TCP RST	Firewall suppression (per-host config)	ICMPv6 responses	Firewall suppression (per-host config)	—	Silent drop (protocol-level, no config required)
OS fingerprinting	Response header variations identify OS	TTL, window size, ICMP format differences	—	Hop Limit, ICMPv6 format differences	—	—	No responses to fingerprint
Parser differential evasion	Firewall and destination disagree on field positions	Variable IHL (20–60 bytes)	—	Extension header chain (unbounded)	—	—	Fixed offsets (no parser exists)

Attack Class	Attack Mechanism	IPv4 Enabler	IPv4 Defense	IPv6 Enabler	IPv6 Defense	IPv4-64 Enabler	IPv4-64 Defense
TCP option manipulation	Middlebox interference with option negotiation	Variable-length TCP options at variable offset	—	Same TCP as IPv4	—	—	No TCP options (WS, SACK-OK are fixed flags)
Connection state probing	RST responses confirm listening state	RST sent on closed ports	—	RST sent on closed ports	—	—	Silent drop on invalid flag combinations
Blind TCP injection	Off-path attacker guesses sequence number	32-bit sequence space	—	Same TCP as IPv4	—	—	SVT blocks spoofed source + Retry Cookie validates handshake
UDP zero-checksum corruption	Silent data corruption on unchecked UDP	Optional checksum (zero = skip)	—	—	Mandatory checksum	—	Mandatory checksum (zero = drop)
Routing loop amplification	Packets circulate indefinitely	TTL field	TTL decrement	Hop Limit field	Hop Limit decrement	TTL field	TTL decrement (ICMP Time Exceeded optional, not required)

Appendix B — Per-Packet Operation Count by Protocol

Each row is one operation the router or host must perform per packet. Operations marked with a clock symbol (🕒) are on the critical forwarding path. Operations marked with a branch symbol (🌿) contain conditional logic.

Operation	IPv4	IPv6	IPv4-64
Read IP version	🕒 Fixed offset (byte 0, upper nibble)	🕒 Fixed offset (byte 0, upper nibble)	🕒 Fixed offset (byte 0, upper nibble)
Determine header length	🕒🌿 Read IHL (byte 0, lower nibble), multiply by 4	🕒 Fixed (40 bytes), but extension headers vary total	🕒 Fixed (32 bytes)
Locate payload start	🕒🌿 Computed from IHL	🕒🌿 Walk extension header chain	🕒 Fixed (byte 32)
Verify header checksum	🕒 One's complement sum over $IHL \times 4$ bytes (10–30 additions)	Not required	Not required
Decrement TTL / Hop Limit	🕒 Subtract 1 at variable offset (byte 8 in minimum header)	🕒 Subtract 1 at fixed offset (byte 7)	🕒 Subtract 1 at fixed offset
Recompute header checksum	🕒 Full recomputation after TTL change	Not required	Not required
Read destination address	🕒 Fixed offset (bytes 16–19)	🕒 Fixed offset (bytes 24–39)	🕒 Fixed offset (bytes 24–31)
Route lookup	🕒 32-bit prefix match	🕒 128-bit prefix match	🕒 64-bit prefix match
Parse IP options	🕒 TLV loop (0–40 bytes, type dispatch per option)	N/A (handled by extension headers)	Not required (no options)
Walk extension header chain	N/A	🕒 Follow Next Header field, unbounded depth	N/A (no extension headers)
Read transport protocol number	🕒 Fixed offset (byte 9)	🕒🌿 Final Next Header in chain (variable offset)	🕒 Fixed offset (byte 7)
Locate transport header	🕒🌿 Byte offset = $IHL \times 4$	🕒🌿 After last extension header (variable)	🕒 Fixed (byte 32)

Operation	IPv4	IPv6	IPv4-64
Read source/destination ports	Variable offset (depends on IHL)	Variable offset (depends on extension chain)	Fixed offset (bytes 32–35)
Read TCP flags	Variable offset ($\text{IHL} \times 4 + 13$)	Variable offset	Fixed offset (bytes 44–45)
Parse TCP options	TLV loop per segment (0–40 bytes)	Same TCP as IPv4	Not required (no TCP options)
Verify SVT	N/A	N/A	Optional (one HMAC comparison at fixed offset)
Verify Fragment Token	N/A	N/A	One comparison at fixed offset (fragmented packets only)
Verify Retry Cookie	N/A	N/A	One HMAC comparison (TCP handshake only)

Total conditional branches in forwarding path:

Protocol	Branches (minimum header)	Branches (maximum header / worst case)
IPv4	4 (version, IHL bounds, checksum, TTL zero)	4 + N (where N = number of IP options + TCP options)
IPv6	3 (version, hop limit zero, next header type)	3 + M (where M = extension header chain depth, unbounded)
IPv4-64	2 (version, payload length)	2 (same — no variable cases)

Appendix C — Cache Line Utilization

Modern CPUs read memory in 64-byte cache lines. A cache line miss costs 4–5 nanoseconds (L2) or 50–100 nanoseconds (main memory). At 100 Gbps with 64-byte minimum packets, the per-packet budget is approximately 5.12 nanoseconds. A single L2 cache miss consumes the entire budget.

Scenario	IPv4 (bytes)	IPv6 (bytes)	IPv4-64 (bytes)	Cache Lines Required
IP header only (minimum)	20	40	32	IPv4: 1, IPv6: 1, IPv4-64: 1
IP header only (maximum)	60	40 + extensions (unbounded)	32	IPv4: 1, IPv6: 1+, IPv4-64: 1
IP + TCP headers (minimum)	40	60	56	IPv4: 1, IPv6: 1, IPv4-64: 1
IP + TCP headers (maximum)	120	100 + extensions	56	IPv4: 2, IPv6: 2+, IPv4-64: 1
IP + UDP headers	28	48	42	IPv4: 1, IPv6: 1, IPv4-64: 1
IP + TCP headers + first 8 bytes payload	48–128	68–108+	64	IPv4: 1–2, IPv6: 2+, IPv4-64: 1

IPv4-64 IP + TCP headers (56 bytes) plus 8 bytes of payload data equals exactly 64 bytes. One cache line contains the complete IP header, the complete TCP header, and the first 8 bytes of application data. The first 8 bytes of payload are significant for protocols like HTTP/2, gRPC, and DNS-over-TCP, where the frame header or message length field is in the first 8 bytes.

Appendix D — Routing Table Prefix Length Comparison

The route lookup is the most time-consuming operation in the forwarding path. Lookup time depends on the address width and the data structure used.

Property	IPv4	IPv6	IPv4-64
Address width	32 bits	128 bits	64 bits
Maximum prefix length	/32	/128	/64
Typical prefix length (global table)	/24	/48	/32 (legacy), up to /40 (extended)
Trie depth (binary trie)	32 levels	128 levels	64 levels
TCAM entry width	32 bits + mask	128 bits + mask	64 bits + mask

Property	IPv4	IPv6	IPv4-64
TCAM entries per unit silicon	Baseline	Approximately $0.25 \times (4 \times \text{wider entries})$	Approximately $0.5 \times (2 \times \text{wider entries})$
Compressed trie nodes	4 bytes per node	16 bytes per node	8 bytes per node

IPv4-64 routing table entries are twice the width of IPv4 entries and half the width of IPv6 entries. A TCAM that holds 1 million IPv4 routes holds approximately 500,000 IPv4-64 routes or approximately 250,000 IPv6 routes. IPv4-64 requires half the TCAM capacity of IPv6 for the same number of routes.

For the legacy address space (upper 32 bits zero), the routing table entry is effectively 32 bits — the upper 32 bits are always zero and can be masked. Existing IPv4 routing table entries carry over without expansion. Only routes to extended addresses require the full 64-bit entry.

Appendix E — SVT Brute-Force Resistance

Parameter	Value
SVT width	32 bits
Possible SVT values	4,294,967,296
Epoch duration (N minutes, configurable)	Typical: 5 minutes
Active epochs (current + previous)	2
Valid SVT values per source address per moment	2 (one per active epoch)
Probability of random match per packet	$2 / 2^{32} \approx 4.66 \times 10^{-10}$
Packets required for 50% probability of one match	$\approx 1.49 \times 10^9$
At 1 Mpps attack rate, time to 50% match probability	≈ 24.8 minutes
At 10 Mpps attack rate, time to 50% match probability	≈ 2.48 minutes
At 100 Mpps attack rate, time to 50% match probability	≈ 14.9 seconds

At 100 million packets per second (approximately 51 Gbps at minimum packet size), an attacker achieves a single valid SVT match in approximately 15 seconds. However, each match is valid for only one specific source address. The attacker must repeat the process for every source address they want to spoof. To spoof 1,000 different source addresses simultaneously at 100 Mpps total, the expected time per valid match per address increases

to approximately 4.1 hours.

The SVT is not designed to resist state-level attackers with unlimited bandwidth. It is designed to make casual spoofing impractical and to raise the cost of volumetric spoofed attacks by orders of magnitude compared to the current cost (zero).

Appendix F — Fragment Token Collision Probability

Parameter	Value
Fragment Token width	16 bits
Identification width	16 bits
Combined guessing difficulty	2^{32} (Identification $\times$ Fragment Token)
Hash function	SipHash-2-4 (keyed, non-cryptographic)
Key width	64 bits (per-connection secret)
Probability of blind match per forged fragment	$1 / 2^{32} \approx 2.33 \times 10^{-10}$
Fragments required for 50% match probability	$\approx 2.97 \times 10^9$
At 1 Mpps, time to 50% match probability	≈ 49.5 minutes

SipHash-2-4 was selected for computational cost, not cryptographic strength. It executes in sub-nanosecond time on modern CPUs. The per-connection secret means that a token valid for one connection is not valid for another. An attacker who observes the token on one connection (on-path) gains no advantage for injecting into a different connection.

Fragment injection requires the target to be receiving fragmented traffic. On the modern internet, fragmented traffic is a small fraction of total traffic. Most paths support 1500-byte MTU. QUIC and HTTP/2 avoid fragmentation by design. The attack applies primarily to legacy UDP applications (DNS with large responses, certain VPN tunnels) that generate fragments.

Appendix G — Retry Cookie Bit Allocation

The 64-bit Retry Cookie field is divided as follows during a SYN-ACK:

Bit Range	Width	Content
63–8	56 bits	Truncated HMAC-SHA256 output
7–5	3 bits	MSS index (8 possible values)
4–1	4 bits	Window scale factor (0–14)
0	1 bit	SACK-OK (0 = not supported, 1 = supported)

MSS Index Mapping

Index (3 bits)	MSS Value (bytes)	Typical Use
0	536	Minimum (RFC 879)
1	1220	IPv6 minimum path MTU minus headers
2	1440	Common behind PPPoE
3	1452	Common behind PPPoE with IPv6
4	1460	Ethernet standard (1500 MTU minus 40-byte TCP/IP)
5	4312	IEEE 802.3 jumbo frame class
6	8960	9000-byte jumbo frame minus headers
7	Reserved	Future use

Window Scale Factor Mapping

Encoded Value (4 bits)	Window Scale Factor	Maximum Window Size
0	0 (no scaling)	65,535 bytes
1	1	131,070 bytes
2	2	262,140 bytes
3	3	524,280 bytes
4	4	1,048,560 bytes
5	5	2,097,150 bytes
6	6	4,194,300 bytes
7	7	8,388,600 bytes
8	8	16,777,200 bytes
9	9	33,554,430 bytes
10	10	67,108,860 bytes
11	11	134,217,720 bytes
12	12	268,435,440 bytes
13	13	536,870,880 bytes
14	14	1,073,741,760 bytes (≈ 1 GB)
15	Reserved	Future use

Comparison to IPv4 SYN Cookie Bit Budget

Parameter	IPv4 SYN Cookie (32-bit ISN)	IPv4-64 Retry Cookie (64-bit field)
Total bits available	32	64
HMAC / hash output	24 bits (after MSS encoding)	56 bits
MSS encoding	3 bits (8 values)	3 bits (8 values)
Window scale	Not encoded (lost)	4 bits (15 values)
SACK capability	Not encoded (lost)	1 bit

Parameter	IPv4 SYN Cookie (32-bit ISN)	IPv4-64 Retry Cookie (64-bit field)
Timestamp	Not encoded (lost)	Timestamp is input to HMAC, not carried in cookie
Collision resistance	$2^{24} \approx 16.7$ million	$2^{56} \approx 7.2 \times 10^{16}$

Appendix H — Validated Flag Amplification Factor Reduction

Current amplification factors for common UDP reflection attacks, and the effect of the Validated flag with a rate-limiting middlebox:

Service	Request Size	Response Size	Amplification Factor	With Validated Flag Rate Limit
DNS (ANY query)	60 bytes	4,000+ bytes	$66 \times$	Unvalidated responses rate-limited to configurable threshold
NTP (monlist)	234 bytes	48,000+ bytes	$205 \times$	Same
SSDP (M-SEARCH)	50 bytes	30,000+ bytes	$600 \times$	Same
Memcached (stats)	15 bytes	750,000+ bytes	$50,000 \times$	Same
CHARGEN	1 byte	200+ bytes	$200 \times$	Same
CLDAP	52 bytes	4,000+ bytes	$76 \times$	Same

The Validated flag does not eliminate amplification. It provides a signal that middleboxes use to throttle unvalidated responses. The rate limit threshold is a policy decision made by the middlebox operator. A threshold of zero (drop all unvalidated UDP responses) eliminates amplification entirely but also drops legitimate first-contact UDP responses. A threshold matched to expected baseline traffic rates throttles attack volume while permitting normal operation.

The flag also creates a classification mechanism for traffic during an active attack. A scrubbing service or upstream provider can separate validated and unvalidated UDP traffic into different queues. Validated traffic passes at full rate. Unvalidated traffic is rate-limited or dropped. This separation does not require the scrubbing service to understand DNS, NTP, or memcached protocols.

Appendix I — CV Flag State Discrimination

The TCP CV (Connection Validated) flag is set after the Retry Cookie handshake completes. It allows firewalls and middleboxes to distinguish established connections from unvalidated connection attempts by reading a single bit at a fixed offset.

Packet Type	SYN Flag	CV Flag	Meaning	Firewall Action
New connection attempt	1	0	Unvalidated SYN	Apply SYN rate limiting, connection throttling
Handshake in progress	0	0	ACK echoing Retry Cookie, not yet verified	Pass to TCP stack for cookie verification
Established connection	0	1	Handshake complete, cookie verified	Fast-path forwarding, skip connection tracking lookup
Data on established connection	0	1	Normal data segment	Fast-path forwarding
Connection teardown	0 (FIN=1)	1	Graceful close on established connection	Fast-path forwarding
Spoofed data injection attempt	0	1	Attacker sets CV without completing handshake	TCP stack verifies: if no matching connection state exists, silent drop

Firewall State Table Impact

A traditional IPv4/IPv6 stateful firewall maintains a connection tracking table. For every TCP connection passing through the firewall, the table stores the 5-tuple, sequence numbers, window size, and connection state (SYN_SENT, ESTABLISHED, FIN_WAIT, etc.). The firewall updates this table on every packet. The table consumes memory proportional to the number of concurrent connections. Under a SYN flood, the table grows rapidly and may exhaust memory.

With the CV flag, the firewall can implement a two-tier architecture:

Tier	CV Flag	Processing
Fast path	CV = 1	5-tuple match against a Bloom filter or hash table. If match: forward. No sequence number tracking, no state update.
Slow path	CV = 0	Full stateful inspection. SYN rate limiting. Retry Cookie verification delegation.

The fast path handles the majority of traffic (established connections) with minimal per-packet cost. The slow path handles only new connections and handshakes. Under a SYN flood, only the slow path is stressed. Established connections continue to be forwarded at full rate on the fast path.

Appendix J — ASIC Gate Count Estimation

The following estimates compare the relative gate count for header processing logic in each protocol. Absolute gate counts depend on the specific ASIC architecture and fabrication process. These ratios are independent of process node.

Logic Block	IPv4	IPv6	IPv4-64
Header length computation	Multiplexer (11 input, IHL values 5–15)	Fixed + extension chain walker (sequential)	Eliminated (constant)
Payload offset computation	Adder ($\text{IHL} \times 4$)	Accumulator (sum of extension header lengths)	Eliminated (constant 32)
Header checksum verification	16-bit one's complement adder tree (10–30 inputs)	Eliminated	Eliminated
Header checksum recomputation	Same adder tree + TTL increment compensation	Eliminated	Eliminated
Option parser	TLV state machine (type dispatch, length validation, boundary check)	Extension header state machine (next-header dispatch, length validation)	Eliminated
Transport header locator	Multiplexer (selects start byte based on IHL)	Output of extension chain walker	Eliminated (constant byte 32)

Logic Block	IPv4	IPv6	IPv4-64
TCP option parser	TLV state machine (type dispatch per segment)	Same (identical TCP)	Eliminated
Total header processing state machines	2 (IP options + TCP options)	2 (extension chain + TCP options)	0
Total multiplexers for variable offsets	3+ (payload start, transport start, TCP data start)	2+ (transport start, TCP data start)	0

Estimated relative gate count for header processing (normalized to IPv4 = 1.0):

Protocol	Relative Gate Count
IPv4	1.0
IPv6	0.8–1.2 (checksum removed, but extension chain walker added)
IPv4-64	0.3–0.4 (all state machines and multiplexers eliminated)

The freed silicon area can be allocated to: deeper routing table TCAM, additional packet buffer memory, more queue scheduling logic, or SVT verification units (one HMAC-SHA256 comparator per port).

Appendix K — Software Processing Cycle Estimates

Estimated CPU cycles per packet for header processing on a modern x86-64 processor (3 GHz, out-of-order execution, branch prediction). These estimates exclude the route lookup (which is identical across protocols) and cover only header validation, field extraction, and checksum operations.

Operation	IPv4 Cycles	IPv6 Cycles	IPv4-64 Cycles
Version check	1	1	1
Header length determination	3 (load IHL, mask, shift, multiply)	1 (constant)	0 (constant, compiled out)
Header checksum verification	12–30 (depends on IHL)	0	0
TTL/Hop Limit decrement	1	1	1
Header checksum recomputation	12–30	0	0
Payload length validation	3	2	2

Operation	IPv4 Cycles	IPv6 Cycles	IPv4-64 Cycles
IP option parsing (common case: no options)	2 (branch on IHL == 5)	0	0
IP option parsing (worst case: 40 bytes options)	40–80 (loop + type dispatch)	N/A	N/A
Extension header walk (common case: none)	N/A	2 (branch on Next Header)	N/A
Extension header walk (worst case: 4 extensions)	N/A	30–60 (loop + dispatch)	N/A
Transport header offset computation	3	3 (after chain walk)	0 (constant, compiled out)
TCP option parsing (common case: timestamp only)	8–12 (loop, 2 iterations)	8–12 (same TCP)	0
TCP option parsing (worst case: full options)	30–50 (loop, multiple iterations)	30–50 (same TCP)	0
Total (common case, TCP)	35–50	15–20	4–5
Total (worst case, TCP)	90–170	60–120	4–5

At 10 million packets per second, the cycle savings for IPv4-64 versus IPv4 common case is approximately 300–450 million cycles per second. On a 3 GHz core, this represents 10–15% of one core’s total capacity freed for application processing or additional packet throughput.

The critical property is the final row: IPv4-64 worst case equals its common case. The processing time variance is zero. This eliminates jitter in software forwarding performance, which matters for real-time applications (VoIP, gaming, financial trading) where consistent latency is more important than average latency.

Appendix L — Epoch Overlap and Key Rotation Timing

The SVT uses two active secrets (current epoch and previous epoch) to handle in-flight packets during key rotation. The timing parameters are:

Parameter	Value	Rationale
Epoch duration (N)	Configurable, typical 5 minutes	Balance between rotation frequency (security) and overlap window (operational tolerance)
Active secrets	2 (current + previous)	Packets stamped in the previous epoch that are still in transit remain valid
Maximum packet lifetime	Defined by TTL (max 255 hops $\times$ ~1ms per hop $\approx$ 255ms typical)	In-flight packets complete transit well within one epoch
Secret generation	CSPRNG, 128 bits	Matches HMAC-SHA256 key size
Rotation trigger	Wall clock crossing epoch boundary	No coordination between routers required — each router rotates independently

Key Rotation Failure Modes

Failure	Effect	Mitigation
Clock skew between stamping router and verifying router	SVT computed with different epoch counter, verification fails	Two-epoch overlap covers skew up to one full epoch duration
Clock skew exceeding one epoch duration	Both active secrets mismatch, all packets from this source dropped	NTP synchronization (already deployed on all production routers)
Secret desynchronization (one peer rotates, other does not update)	Verification failures on all packets from the desynced peer	Secrets are local — each router generates its own. Only the bilateral sharing must stay synchronized. Monitoring alerts on elevated SVT failure rates.
Router reboot during epoch	New secrets generated, old in-flight packets fail verification	Two-epoch overlap covers the transition. Reboots take longer than maximum packet lifetime.

Appendix M — Bilateral SVT Trust Topology Examples

Example 1: Two-Peer Exchange

Operator A and Operator B peer at an internet exchange point. They exchange SVT secrets. Each stamps outgoing packets. Each verifies incoming packets from the other.

Direction	Stamping Router	Verifying Router	SVT Secret Used
A → B	A's edge router	B's edge router	A's secret (shared with B)
B → A	B's edge router	A's edge router	B's secret (shared with A)

Operator C does not participate. Packets from C arrive at A or B with unverifiable SVTs. A and B apply their policy (pass, rate-limit, or drop) to unverified traffic.

Example 2: Transit Provider Chain

Operator A is a customer of Transit Provider T. T peers with Operator B. A shares its SVT secret with T. T shares its SVT secret with B.

Path	Stamping Router	Verifying Router	SVT Secret
A → T → B	A's edge router	T can verify (has A's secret). B cannot verify A's SVT (does not have A's secret). B can verify T's transit stamp if T re-stamps.	A's secret known to T only

Two models exist for transit:

Transparent transit: T forwards A's SVT unchanged. B cannot verify it. B treats it as unverified traffic from a non-participating network. A's SVT provides value only to T.

Re-stamping transit: T verifies A's SVT on ingress, then re-stamps the packet with T's own SVT on egress. B verifies T's SVT. B knows the packet entered the internet through T, which attested the source. B does not know whether T verified the original source. This is equivalent to BCP 38 trust: B trusts T to filter its customers.

The re-stamping model is the practical one. It matches the existing trust hierarchy: customers trust their transit provider, and peers trust each other. The SVT makes this trust verifiable per-packet.

Example 3: Major Transit Provider Mesh

If the 10 largest transit providers (by global traffic volume) implement bilateral SVT exchange:

Participating Providers	Bilateral Secrets	Verified Traffic Coverage (approximate)
Top 5	10 bilateral pairs	Majority of inter-provider backbone traffic
Top 10	45 bilateral pairs	Majority of global internet traffic
Top 20	190 bilateral pairs	Near-total coverage of traffic transiting Tier 1/2 providers

45 bilateral secret exchanges require 45 configuration changes per provider. This is comparable to the number of BGP peering sessions a Tier 1 provider already maintains. The operational overhead is marginal.

Appendix N — Protocol Feature Adoption Lineage

This table traces the origin of each IPv4-64 feature to the protocol or practice that first introduced or validated it.

IPv4-64 Feature	Origin	Year	How IPv4-64 Differs
Flow Label (20 bits)	IPv6 (RFC 2460)	1998	Unchanged in function. Placed at fixed offset in a smaller header.
No header checksum	IPv6 (RFC 2460)	1998	Same rationale: link and transport checksums are sufficient.
Payload Length (vs Total Length)	IPv6 (RFC 2460)	1998	Same rationale: removes dependency on header length.
Mandatory UDP checksum	IPv6 (RFC 2460, enforced in RFC 6936)	1998/2013	Same rationale: prevents silent corruption.

IPv4-64 Feature	Origin	Year	How IPv4-64 Differs
Stateless retry handshake	QUIC (RFC 9000)	2021	Applied to TCP as a dedicated field. QUIC uses it for QUIC connections only.
SYN cookies	D.J. Bernstein	1996	Replaced by Retry Cookie. Dedicated field eliminates sequence number overloading.
Ingress filtering concept	BCP 38 (RFC 2827)	2000	Encoded in-packet as SVT. Voluntary ACL configuration replaced by cryptographic stamp.
SipHash for fast keyed hashing	Jean-Philippe Aumasson, Daniel J. Bernstein	2012	Used for Fragment Token. Selected for speed (sub-nanosecond per computation).
HMAC-SHA256 for authentication	RFC 4868	2007	Used for SVT and Retry Cookie. Standard construction, truncated to required output width.
Fixed header with no options	Design principle (new)	2026	No prior deployed protocol eliminates all variable-length parsing from IP, TCP, and UDP simultaneously.
Silent drop semantics	Partial in various firewall configurations	Varies	Elevated from implementation choice to protocol-level requirement.

IPv4-64 Feature	Origin	Year	How IPv4-64 Differs
Source-only fragmentation	IPv6 (RFC 8200)	2017	Not adopted. IPv4-64 preserves transit fragmentation with Fragment Token protection instead.
Extension header chain	IPv6 (RFC 2460)	1998	Not adopted. Identified as source of firewall evasion and parser complexity.
Mandatory IPsec	IPv6 (RFC 2460, relaxed in RFC 6434)	1998/2011	Not adopted. Encryption is an application/transport-layer decision.
Neighbor Discovery	IPv6 (RFC 4861)	2007	Not adopted. ARP extended for 64-bit addresses is simpler.
SLAAC	IPv6 (RFC 4862)	2007	Not adopted. DHCP provides stronger administrative control.

Appendix O — Strict Drop Condition Cross-Reference

Each drop condition from the companion specification [[HOWL-NET-1-2026](#)] Appendix G is listed here with the specific attack it prevents and the IPv4/IPv6 behavior for the same condition.

#	Condition	Attack Prevented	IPv4 Behavior	IPv6 Behavior	IPv4-64 Behavior
1	Payload Length mismatch	Buffer overflow, truncation attacks	Total Length mismatch: implementation dependent	Payload Length mismatch: drop (RFC 8200)	Silent drop

#	Condition	Attack Prevented	IPv4 Behavior	IPv6 Behavior	IPv4-64 Behavior
2	Version field wrong	Protocol confusion	Drop or process as different version	Drop	Silent drop
3	Reserved bits nonzero	Covert channels, protocol probing	Ignored (Postel's Law)	Ignored for some fields	Silent drop
4	Fragment Token mismatch	Fragment injection	N/A (no Fragment Token)	N/A (source-only fragmentation)	Drop all fragments in group
5	SVT invalid	Source spoofing	N/A (no SVT)	N/A (no SVT)	Silent drop
6	Retry Cookie invalid	SYN flood, connection spoofing	N/A (SYN cookies in sequence number)	N/A (same TCP)	Silent drop
7	TTL zero	Routing loop detection	ICMP Time Exceeded (required)	ICMPv6 Time Exceeded (required)	Silent drop (ICMP optional)
8	Protocol field unknown	Service probing	ICMP Protocol Unreachable	ICMPv6 Parameter Problem	Silent drop
9	Invalid TCP flag combination	Scanner fingerprinting (Xmas scan, null scan)	Implementation dependent (some respond, some drop)	Implementation dependent	Silent drop
10	Fragment overlap	Ambiguous reassembly exploitation	Implementation dependent (varies by OS)	Drop (RFC 8200 Section 4.5)	Drop entire fragment group
11	Packet smaller than 32 bytes	Malformed packet exploits	Implementation dependent	Drop (below minimum IPv6 header)	Silent drop

#	Condition	Attack Prevented	IPv4 Behavior	IPv6 Behavior	IPv4-64 Behavior
12	Packet smaller than transport minimum	Truncated header probing	Implementation dependent	Implementation dependent	Silent drop
13	Checksum mismatch	Corruption, forged segments	Drop (TCP), implementation-dependent (UDP)	Drop	Silent drop
14	UDP checksum zero	Silent corruption	Accepted (checksum optional)	Drop (mandatory since RFC 6936)	Silent drop
15	Address octet outside 0–255	Parse confusion	N/A (binary encoding, not parsed as text at wire level)	N/A	Reject at parse time
16	Address fewer than 4 or more than 8 octets	Parse confusion	N/A (fixed 4 octets)	N/A	Reject at parse time

The column “IPv4 Behavior: Implementation-dependent” identifies conditions where different IPv4 implementations behave differently. These inconsistencies are the source of parser differential attacks (Appendix E of the main analysis, Section 3.6). IPv4-64 eliminates implementation variance by specifying one behavior for every condition: silent drop.

Appendix P — Bandwidth Savings at Scale

Cumulative header size reduction of IPv4-64 versus IPv6 across traffic profiles.

Traffic Profile	Packets per Second	IPv6 IP+Transport Header	IPv4-64 IP+Transport Header	Savings per Packet	Savings per Second	Savings per Day
VoIP (G.711, UDP)	50 pps per call	48 bytes	42 bytes	6 bytes	300 bytes/call	25.9 MB/call
100,000 concurrent VoIP calls	5,000,000 pps	48 bytes	42 bytes	6 bytes	30 MB/s	2.59 TB
Online gaming (UDP, 64-byte payload)	60 pps per player	48 bytes	42 bytes	6 bytes	360 bytes/player	31.1 MB/player
1,000,000 concurrent game sessions	60,000,000 pps	48 bytes	42 bytes	6 bytes	360 MB/s	31.1 TB
IoT telemetry (UDP, 32-byte payload)	1 pps per device	48 bytes	42 bytes	6 bytes	6 bytes/device	518 KB/device
10 billion IoT devices	10,000,000,000 pps	48 bytes	42 bytes	6 bytes	60 GB/s	5.18 PB
HTTP/2 ACK-heavy traffic (TCP, small frames)	1,000,000 pps per server	60 bytes	56 bytes	4 bytes	4 MB/s	345.6 GB
Bulk data transfer (TCP, 1500-byte MTU)	81,274 pps per Gbps	60 bytes	56 bytes	4 bytes	325 KB/s/Gbps	28.1 GB/Gbps

The savings are most significant on small-packet workloads (VoIP, gaming, IoT) where the header constitutes a large fraction of the total packet. On bulk data transfers, the savings are proportionally small but accumulate at datacenter scale.

References

- RFC 791: Internet Protocol, September 1981
- RFC 879: TCP Maximum Segment Size and Related Topics, November 1983
- RFC 2460: Internet Protocol, Version 6 (IPv6) Specification, December 1998
- RFC 2827 (BCP 38): Network Ingress Filtering, May 2000
- RFC 4861: Neighbor Discovery for IP version 6 (IPv6), September 2007
- RFC 4862: IPv6 Stateless Address Autoconfiguration (SLAAC), September 2007
- RFC 4868: Using HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512 with IPsec, May 2007
- RFC 6434: IPv6 Node Requirements, December 2011
- RFC 6936: Applicability Statement for the Use of IPv6 UDP Datagrams with Zero Checksums, April 2013
- RFC 8200: Internet Protocol, Version 6 (IPv6) Specification, July 2017
- RFC 9000: QUIC: A UDP-Based Multiplexed and Secure Transport, May 2021
- Aumasson, J.-P., Bernstein, D.J., “SipHash: a fast short-input PRF,” 2012
- Bernstein, D.J., “SYN cookies,” 1996
- Ptacek, T., Newsham, T., “Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection,” 1998
- Valkó, A., Turányi, Z., “4+4: An Alternative Approach to IPv4 Address Exhaustion,” SIGCOMM CCR, 2002
- draft-chimiak-enhanced-ipv4-03: IPv4 with 64 bit Address Space (EnIP), expired 2016
- draft-omar-ipv10-13: Internet Protocol version 10 (IPv10), expired 2021
- CAIDA Spoofer Project, <https://spoofer.caida.org/>
- [HOWL-NET-1-2026]: IPv4-64: A 64-Bit In-Place Upgrade to IPv4 with Modern Security and Performance, October 2026

[HOWL-NET-2-2026] IPv4-64: Security and Performance Analysis. Github: <https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-NET-2-2026>

[HOWL-NET-1-2026] IPv4-64: A 64-Bit In-Place Upgrade to IPv4 with Modern Security and Performance. Github: <https://github.com/ghowland/papers/tree/main/papers/NET/HOWL-NET-1-2026>